



ساختمان داده‌ها و الگوریتم‌ها

تمرین دوم - ساختارهای داده

پریسا یحیی‌پور، سپنتا غنودی

تاریخ تحویل: ۱۴۰۴/۰۸/۱۳

نمره ۱۵

۱. جنگ داخلی

در دانشکده برق و کامپیوتر ACM و IEEE قصد برگزاری رویداد Maze را دارند. آن‌ها برای انجام تدارکات یک جلسه تشکیل داده و به ترتیب در صف نشسته‌اند. ترتیب نشستن اعضا به شما داده شده. از آنجایی که این دو انجمن با هم اختلاف نظر دارند، هر یک از اعضای این دو انجمن قصد دارند ایده‌های انجمن خودشان را برای این رویداد تصویب کنند. بالاخره رای‌گیری فرا می‌رسد و افراد به نوبت حق دارند یکی از کارهای زیر را به نوبت و با شروع از نفر اول انجام دهند:

۱. یک عضو می‌تواند حق رای دادن عضوی دیگر را برای این دور و دورهای بعدی رای‌گیری را از او بگیرد. در واقع درجا اخراجش کند.
۲. اگر عضوی که نوبت او است ببیند که تمام اعضای باقیمانده از انجمن خودش هستند می‌تواند نظر گروه خود را تصویب کند و جلسه به اتمام می‌رسد.

تا زمانی که یکی از انجمن‌ها بتواند رای خود را تصویب کند افراد به نوبت یکی از کارهای بالا را انجام می‌دهند و اگر به انتهای ردیف برسند، دوباره از ابتدای صف شروع می‌کنند. اگر هر کدام از اعضا به قدری باهوش باشند که بهترین استراتژی را برای برد انجمن خود انتخاب کنند، الگوریتمی ارائه دهید که به ازای هر چینه‌ش گروه برنده را مشخص کند. (اگر n تعداد اعضای جلسه باشد جواب شما باید از $O(n)$ باشد).

پاسخ:

از دو صف استفاده می‌کنیم. از ابتدای ردیف شروع می‌کنیم و افراد عضو IEEE را در صف اول و افراد عضو ACM را در صف دوم به ترتیب enqueue می‌کنیم. (به ازای هر فرد شماره نوشتن در ردیف را در صف قرار می‌دهیم). سپس هر بار ابتدای هر صف را با صف دیگر مقایسه می‌کنیم، کسی که نوبتش زودتر از دیگری باشد می‌تواند حق رای او را بگیرد بنابراین کسی که نوبتش دیرتر است از ابتدای صف dequeue می‌شود و فرد دیگر که در ابتدای صف دیگر بود، از ابتدای صف خارج و با نوبت جدید (n + نوبت کنونی) به انتهای صف اضافه می‌شود تا در دور بعد در صورت نیاز بتواند در رای‌گیری شرکت کند. این فرآیند را تا زمانی که یکی از صف‌ها خالی شود ادامه می‌یابد. هر انجمنی که صف آن زودتر خالی شود یعنی رای‌گیری را باخته و گروه دیگر برنده رای‌گیری شده است. از آنجایی که در این فرآیند هر بار یک نفر از یکی از این دو صف حذف می‌شود پس در بدترین حالت این فرآیند n بار انجام می‌شود که یعنی الگوریتم از $O(n)$ است.

نمره ۲۰

۲. سولات سپنتا

سپنتا در حال آماده شدن برای آزمونک ساختمان داده است و تعداد $n - k + 1$ سوال دارد. او برای حل سوالاتش وارد سایت قدیم می‌شود و تعداد n نفر از دانشجویان سال بالایی اش را می‌بیند که همگی درس ساختمان داده را با نمره‌های متمایز گذرانده‌اند. سپنتا با استراتژی عجیبی از سال بالایی‌هایش سوالات را می‌پرسد. برای پرسیدن سوال اول، سپنتا در میان افراد ۱ تا k از کسی که بیشترین نمره را دارد سوالش را می‌پرسد. سوال دوم را از کسی که بیشترین نمره را بین افراد ۲ تا $k + 1$ ام دارد می‌پرسد و الگوریتمی با پیچیدگی زمانی $O(n)$ طراحی کنید که با گرفتن آرایه نمرات سال بالایی‌ها به سپنتا بگوید هر یک از $n - k + 1$ سوالش را باید از چه کسی پرسد.

پاسخ:

می‌خواهیم روی یک آرایه پیمایش کنیم و ماکسیمم هر پنجره به طول k را بیابیم. از یک صف دو طرفه (deque) استفاده می‌کنیم. این صف این گونه است که می‌توان هم به ابتدا و هم به انتهای آن عنصر با $O(1)$ اضافه و حذف نمود. حال روی عناصر آرایه پیمایش می‌کنیم و صف را به این صورت می‌سازیم.

- اندیس‌های موجود در صف از ابتدا به انتها به صورت صعودی باشند
 - نمرات دانشجویان در صف از ابتدا به انتها نزولی باشد.
- مزیت این پیاده‌سازی این است که هر بار می‌توانیم با $O(1)$ به عنصر با بالاترین نمره در صف دو طرفه دسترسی پیدا کنیم چراکه این عنصر همان عنصر ابتدای صف است. برای اجرای این دو ویژگی به این صورت عمل می‌کنیم. هر بار که عنصر جدیدی می‌بینیم:
- تا زمانی که نمره عنصر جدید از عنصر انتهایی صف بالاتر است، عنصر انتهایی صف را حذف می‌کنیم و پس از حذف شدن عناصر با نمره پایین‌تر، عنصر جدید را قرار می‌دهیم. با این کار ترتیب صعودی نمرات در صف دو طرفه حفظ می‌شود.
 - تا زمانی که اندیس عناصر ابتدای صف از $k - i$ کوچکتر است، آنها را از صف حذف کن. (تا پنجره به طول k حفظ شود).
- بدین ترتیب پس از k بار انجام این فرآیند، ماکسیمم پنجره اول و بعد از به ازای هر بار اضافه کردن عنصر جدید ماکسیمم پنجره بعدی را می‌توان یافت.
- با توجه به اینکه هر دانشجو دقیقاً یک بار به صف اضافه و حداکثر یک بار حذف می‌شود پس الگوریتم در زمان $O(n)$ قابل اجراست.

مثال

در این مثال، آرایه نمرات $A = [10, 5, 12, 8, 15, 7, 9, 11]$ و اندازه پنجره $k = 3$ است. مراحل اجرای الگوریتم به شرح زیر است:

- مرحله ۱ (اندیس ۰، نمره ۱۰): Deque خالی است، پس اندیس ۰ به آن اضافه می‌شود.
وضعیت Deque: $[0]$
- مرحله ۲ (اندیس ۱، نمره ۵): نمره ورودی (۵) از نمره آخرین عضو Deque یعنی $A[0]$ (۱۰) کمتر است. پس اندیس ۱ به انتها اضافه می‌شود.
وضعیت Deque: $[0, 1]$
- مرحله ۳ (اندیس ۲، نمره ۱۲): نمره ورودی (۱۲) از نمره $A[1]$ (۵) بیشتر است، پس ۱ حذف می‌شود. سپس از نمره $A[0]$ (۱۰) نیز بیشتر است، پس ۰ هم حذف می‌شود. در نهایت اندیس ۲ اضافه می‌شود.
وضعیت Deque: $[2]$
- مرحله ۴ (اندیس ۳، نمره ۸): خروجی: ماکسیمم پنجره قبلی $[10, 5, 12]$ برابر با $A[2]$ یعنی ۱۲ است. پردازش: نمره ۸ از $A[2]$ (۱۲) کمتر است، پس اندیس ۳ اضافه می‌شود.
وضعیت Deque: $[2, 3]$
- مرحله ۵ (اندیس ۴، نمره ۱۵): خروجی: ماکسیمم پنجره قبلی $[5, 12, 8]$ برابر با $A[2]$ یعنی ۱۲ است. پردازش: نمره ۱۵ از $A[3]$ (۸) و سپس از $A[2]$ (۱۲) بزرگتر است، پس هر دو حذف شده و اندیس ۴ اضافه می‌شود.
وضعیت Deque: $[4]$
- مرحله ۶ (اندیس ۵، نمره ۷): خروجی: ماکسیمم پنجره قبلی $[12, 8, 15]$ برابر با $A[4]$ یعنی ۱۵ است. پردازش: نمره ۷ از $A[4]$ (۱۵) کمتر است، پس اندیس ۵ اضافه می‌شود.
وضعیت Deque: $[4, 5]$
- مرحله ۷ (اندیس ۶، نمره ۹): خروجی: ماکسیمم پنجره قبلی $[8, 15, 7]$ برابر با $A[4]$ یعنی ۱۵ است. پردازش: نمره ۹ از $A[5]$ (۷) بیشتر است، پس ۵ حذف می‌شود. سپس اندیس ۶ اضافه می‌شود.
وضعیت Deque: $[4, 6]$

۸. مرحله ۸ (اندیس ۷، نمره ۱۱): خروجی: ماکسیمم پنجره قبلی [۹, ۷, ۱۵] برابر با $A[4]$ یعنی ۱۵ است. پردازش: اندیس ابتدای صف (۴) چون از پنجره فعلی [۵, ۷] خارج شده، حذف می‌شود. سپس نمره ۱۱ از $A[6]$ (۹) بیشتر است، پس ۶ نیز حذف شده و اندیس ۷ اضافه می‌شود. وضعیت: Deque [۷]

۹. خروجی نهایی: ماکسیمم آخرین پنجره [۱۱, ۹, ۷] برابر با $A[7]$ یعنی ۱۱ است.

۳. پشته احساسات

۱۵ نمره

در ذهن رایلی پنج احساس اصلی وجود دارد: خشم، شادی، غم، ترس و نفرت. می‌خواهیم محاسبه کنیم که هر کدام از این احساسات چه زمانی ذهن رایلی را کنترل می‌کنند. هر احساس زمانی که شروع به کار می‌کند، ساعت را یادداشت می‌کند و زمانی که کارش تمام شد، بازهم ساعت پایان را در دفتر ذهن رایلی می‌نویسد. هر احساسی که کنترل ذهن رایلی را به دست می‌گیرد، کار احساس قبلی را موقتاً متوقف می‌کند. فرض کنید در دفتر ذهن رایلی n رکورد از ساعت شروع و پایان احساسات وجود دارد. الگوریتمی از زمان $O(n)$ به کمک پشته ارائه کنید که محاسبه کند هر احساس چقدر کنترل ذهن رایلی را در دست داشته است. همچنین حافظه اضافه مصرف شده باید از $O(n)$ باشد. توجه داشته باشید که هیچ دو احساسی همزمان شروع به کار نمی‌کنند و همزمان خاتمه نمی‌یابند.

مثال: در دفتر ذهن رایلی این رکوردها وجود دارد:

احساس	شروع/پایان	زمان فعلی
شادی	شروع	۰
غم	شروع	۲
غم	پایان	۵
شادی	پایان	۶

شادی در زمان صفر کنترل ذهن رایلی را به دست گرفته، در زمان دو غم وارد عمل می‌شود درحالی که شادی هنوز کارش تمام نشده است. بنابراین شادی به طور موقت متوقف می‌شود تا غم ذهن رایلی را کنترل کند. در زمان ۵ کار غم به پایان رسیده بنابراین غم به مدت ۳ واحد زمانی ذهن رایلی را کنترل کرده است. سپس شادی دوباره کنترل ذهن رایلی را به دست می‌گیرد پ در زمان ۶ کارش تمام می‌شود. بنابراین مدت زمانی که شادی ذهن رایلی را کنترل کرده برابر ۳ واحد زمانی است (از زمان ۰ تا ۲ برابر ۲ واحد زمانی و از زمان ۵ تا ۶ برابر ۱ واحد زمانی)

پاسخ:

ابتدا احساس اول از دفتر ذهن را در پشته قرار می‌دهیم. همچنین زمان جاری را دنبال می‌کنیم و از یک آرایه استفاده می‌کنیم، به طوری که هر عنصر آن زمان اختصاصی صرف شده توسط احساس تا زمان جاری را نگه می‌دارد (برای راحتی می‌توان هر ایندکس از آرایه را به یک احساس اختصاص داد). سپس به احساس بعدی در دفتر می‌رویم. زمان شروع/پایان احساس بعدی واضحاً بزرگ‌تر از زمان شروع احساس روی پشته است. مستقیماً از اختلاف بین زمان رکورد احساس بعدی و زمان رکورد احساس فعلی استفاده کنیم تا زمان اختصاصی احساس فعلی که سر پشته است را به روز کنیم. در این نقطه، کنترل از احساس قبلی به احساس جدید منتقل می‌شود؛ یا به دلیل شروع احساس جدید یا به دلیل پایان احساس قبلی. بنابراین دیگر نمی‌توانیم با همان احساس قبلی ادامه دهیم. اگر رکورد بعدی در دفتر مربوط به شروع یک احساس باشد، آن را به پشته اضافه می‌کنیم. اما اگر مربوط به پایان یک احساس باشد یعنی احساس سر پشته به پایان رسیده است. همچنین باید حواسمان باشد در صورتی که رکورد فعلی مربوط به پایان یک احساس است، باید ابتدا زمان اختصاصی احساس قبلی را افزایش دهیم و سپس آن را از پشته pop کنیم. در نهایت آن آرایه زمان اختصاصی هر احساس را نشان می‌دهد.

۴. قطعه قطعه جمع گردد، وانگهی سورت شود

۱۵ نمره

یک آرایه داریم که در واقع یک جایگشت از اعداد ۱ تا n است. می‌خواهیم آرایه را به تعدادی زیر آرایه (هر زیر آرایه از حذف یک پیشوند و پسوند که می‌توانند تهی باشند ساخته می‌شوند) تقسیم کنیم و هر زیر آرایه را جداگانه مرتب کنیم. در نهایت با اتصال این بخش‌های مرتب شده به آرایه نهایی می‌رسیم که مرتب شده است. الگوریتمی به کمک پشته ارائه کنید که مشخص کند برای رسیدن به این هدف، آرایه باید به چند زیر آرایه تقسیم شود. الگوریتم شما باید $O(n)$ باشد و حافظه اضافه استفاده شده نیز $O(n)$ باشد.

پاسخ:

ایده اصلی این روش این است که اگر عددی در آرایه از هر عددی در بخش‌های قبلی کوچک‌تر باشد، نمی‌تواند یک بخش جدید بسازد، چون ما اجازه نداریم عناصر بخش‌های مختلف را جابه‌جا کنیم تا ترتیبشان درست شود. بنابراین، هنگام پیمایش آرایه، از یک پشته استفاده می‌کنیم تا بیشترین مقدار هر بخشی که تا آن لحظه ساخته‌ایم را در آن نگه داریم. از ابتدای آرایه شروع به بررسی می‌کنیم. باید تصمیم بگیریم که آیا عدد فعلی می‌تواند شروع کننده یک بخش جدید باشد یا خیر. بنابراین دو حالت پیش می‌آید:

۱. عدد فعلی از عدد سر پشته بزرگ‌تر است: می‌تواند یک بخش جدید تشکیل دهد. این عدد را به پشته اضافه می‌کنیم
۲. عدد فعلی از سر پشته کوچک‌تر است: نمی‌تواند بخش جدید بسازد پس باید ادغام شود. برای ادغام تمام بخش‌هایی که مقدار بیشینه آنها از عدد فعلی بزرگ‌تر است را از پشته حذف می‌کنیم. سپس بیشترین مقدار بین آنها را انتخاب می‌کنیم و دوباره به پشته اضافه می‌کنیم. در واقع این بخش‌ها با هم ادغام می‌شوند

۵. فشرده سازی عجیب

۱۵ نمره

یکی از دانشجویان نوورودی مهندسی کامپیوتر در حال طرح یک الگوریتم فشرده سازی برای رشته‌ها است. او الگوریتم خود را این گونه تعریف می‌کند: (و البته بعداً مشخص شد که الگوریتم او کاربردی ندارد!)

او یک رشته به طول n به عنوان ورودی الگوریتم می‌گیرد و تمام حروف تکراری رشته را به طوری حذف می‌کند که:

۱. از هر حرف حروف الفبا تنها یک حرف در رشته باقی بماند و باقی تکرارهای آن حرف از رشته حذف شوند.
۲. رشته نهایی بین رشته‌هایی که شرط اول را دارند از لحاظ لغت‌نامه‌ای کمینه باشد: درباره ترتیب لغت‌نامه‌ای اینجا بخوانید.

راه حلی ارائه دهید که او بتواند خروجی این الگوریتم را در $O(n)$ بدست آورد. مثال:

aaaaabbbbcccd ⇒ abcd

adddbbbbcccd ⇒ abcd

azbcbccz ⇒ abcz

پاسخ:

برای حل این مسئله از یک استک و دو آرایه کمکی استفاده می‌کنیم. این دو آرایه به طول تعداد حروف الفبا می‌سازیم و در یکی آخرین ایندکس حضور یک حرف را مشخص می‌کنیم (آرایه *last_index*) و دیگری ابتدا تمام مقادیرش را false می‌گذاریم و هر بار که مقداری را در پشته قرار می‌دهیم، مقدار مربوطه در آرایه را true و وقتی از پشته حذف می‌کنیم مقدار مربوطه را false می‌کنیم (آرایه *seen*). می‌دانیم عملیات‌های ذکر شده روی این دو آرایه از $O(1)$ خواهند بود. حال سراغ پیمایش رشته می‌رویم و از ابتدا شروع به پیمایش می‌کنیم. در هر مرحله ابتدا مقدار حرف جدید را در آرایه *seen* بررسی می‌کنیم. اگر مقدار آن درست بود یعنی قبلاً دیده‌ایم و در استک موجود است و به سراغ حرف بعدی می‌رویم.

در غیر این صورت در هر مرحله، حرف جدید را با حرف بالای استک مقایسه می‌کنیم. اگر از آن بزرگتر بود در استک قرار داده و مقدار مربوطه در آرایه *seen* را true می‌کنیم. اما اگر حرف جدید کوچکتر از حرف بالای استک بود دو حالت به وجود می‌آید:

- این حرف بعداً دوباره در رشته تکرار شده که یعنی اکنون باید از استک حذف شود (که این را از طریق آرایه *last_index* متوجه می‌شویم) و باید مقدار آن در آرایه *seen* به false تغییر یابد و دوباره شروع به مقایسه حرف جدید بالای استک با حرف جدید می‌کنیم.
- این حرف بعداً در رشته تکرار نشده که یعنی باید حرف جدید را روی آن در استک اضافه کنیم.

در آخر وقتی کل کلمات رشته را پیمایش کردیم، مقادیر باقی‌مانده در استک را یکی یکی pop می‌کنیم و در رشته‌ای ذخیره می‌کنیم. برعکس این رشته جواب سوال ماست. با توجه به اینکه ما هر حرف را یک بار پیمایش می‌کنیم و عملیات انجام شده برای آن حرف از $O(1)$ است پس در کل جواب در $O(n)$ خواهد بود.

۶. پک‌من

۲۰ نمره

پک‌من در مسیر مستقیمی حرکت می‌کند. هر خانه از این مسیر مقدار انرژی خودش را دارد. انرژی هر خانه می‌تواند مثبت یا منفی باشد. خانه‌های مثبت انرژی پک‌من را افزایش می‌دهند و خانه‌های منفی انرژی پک‌من را کاهش می‌دهند. پک‌من برای عبور از این مرحله باید کوتاه‌ترین مسیر پیوسته را طی کند به گونه‌ای که انرژی او حداقل برابر K شود. فرض کنید در ابتدای مسیر انرژی اولیه پک‌من برابر صفر است. الگوریتمی با استفاده از صف ارائه کنید که نقاط شروع و پایان کوتاه‌ترین مسیر با انرژی حداقل K را پیدا کند (اگر چند جواب وجود داشتند یکی را به دلخواه پیدا کند). پیچیدگی زمانی الگوریتم شما باید $O(n \log n)$ باشد و حافظه اضافه مصرفی آن $O(n)$ باشد. توجه کنید که n برابر تعداد خانه‌های مسیر است.

مثال: مسیر به این شکل است: $[۹۵, ۴۰, ۳۲, -۳۷, ۸۴]$ و $K = ۱۶۷$ کوتاه‌ترین مسیری که حداقل انرژی آن برابر ۱۶۷ باشد برابر $[۳۲, ۴۰, ۹۵]$ است که طولش ۳ است.

پاسخ:

از آنجایی که محدودیت زمانی داریم نمی‌توان تمام زیر مسیرهای ممکن را بررسی کرد. در واقع مسیر مانند یک آرایه به طول n است. برای حل این سوال از $prefix\ sum$ استفاده می‌کنیم تا مراحل تکراری پیدا کردن مجموع زیر مسیر را حذف کنیم. برای نگهداری این $sum\ prefix$ ها از یک آرایه دیگر به طول n استفاده می‌کنیم. $prefix\ sum$ یعنی مجموع عناصر آرایه از اندیس صفر تا اندیس i که در آرایه اضافه ذخیره می‌کنیم. کمکی که این آرایه جدید به ما می‌کند این است که اگر بخواهیم مجموع عناصر آرایه از j تا i را پیدا کنیم، کافی است:

$$PrefixSum[i] - PrefixSum[j]$$

برای پیدا کردن طول کوچک‌ترین زیر مسیر از صف اولویت‌دار استفاده می‌کنیم.

روند الگوریتم:

- از ابتدای آرایه شروع می‌کنیم و در هر مرحله $sum\ prefix$ را محاسبه می‌کنیم
- اگر $PrefixSim[i]$ از k بیشتر باشد، یک مسیر معتبر داریم که از ابتدای آرایه شروع می‌شود
- صف اولویت‌دار شامل جفت‌های $prefix\ sum$ و اندیس است. این صف به صورت صعودی بر اساس $prefix\ sum$ نگهداری می‌شود.
- صف اولویت‌دار در واقع کوچک‌ترین $prefix\ sum$ را می‌تواند مشخص کند که این عدد اول صف است. اما چرا کوچک‌ترین را می‌خواهیم؟ چون هر چه اختلاف $PrefixSum[i] - PrefixSum[j]$ کوچک‌تر باشد، حاصل نهایی بزرگ‌تر خواهد بود.
- در هر اندیس بعد از محاسبه $prefix\ sum$ مربوط به آن، اختلاف آن را با کوچک‌ترین $prefix\ sum$ در صف بررسی می‌کنیم. اگر بزرگ‌تر از k باشد، یعنی از $j + 1$ تا i یک زیر آرایه مناسب داریم (فرض کنید j اندیس کوچک‌ترین $prefix\ sum$ است و در حال حاضر در حال بررسی اندیس i هستیم). برای محاسبه طول هم داریم: $i - j$. اگر قبلاً جوابی پیدا کرده‌ایم، طول جدید را مقایسه می‌کنیم. اگر کوتاه‌تر بود جواب جایگزین می‌شود. سپس $PrefixSum[j]$ را از صف حذف می‌کنیم. در نهایت $PrefixSum[i]$ را به همراه i به صف اضافه می‌کنیم تا برای محاسبه‌های بعدی استفاده کنیم.
- زمانی که به انتهای آرایه رسیدیم، یا طول کوتاه‌ترین مسیر پیدا شده یا اینکه مسئله جواب ندارد.